JOHANNES KEPLER
UNIVERSITY LINZ

Hannes Brantner,
K1614466
Institute for Machine Learning

@ k01614466@students.jku.at

January 7, 2026

Dense Confidence: Reimplementing and Extending LaSeR with Trajectory-Wide Self-Monitoring for LLM Reasoning

Abstract We present **Dense Confidence**, a novel extension of the LaSeR framework (Yang et al., 2025) designed to address the sparsity of reward signals in logical reasoning tasks. Unlike prior approaches that rely on a scalar confidence score at the sequence termination, our architecture compels the policy to estimate the validity of its reasoning trace at *every* token step. By overloading a specialized control token (the confidence token) and applying a scaled sigmoid projection, we map latent representations to interpretable probabilities.

We evaluate this method on a filtered subset of the `DeepMath-103K` dataset using a 0.5B parameter policy model. While the proposed dense confidence mechanism is agnostic to the specific reinforcement learning algorithm, we employ Group Relative Policy Optimization (GRPO) for our experiments. We define three experimental conditions, consisting of a Baseline, an Auxiliary-Loss-Only variant, and a Self-Rewarding configuration, to rigorously isolate the contribution of dense intrinsic rewards. This work provides a comprehensive analysis of the system architecture, the integration of intrinsic signals into the advantage function, and the implications for scalable oversight in resource-constrained environments. The code is available at https://github.com/Oidlichtnwoada/rl_for_llms.

Course: Practical Work in AI
Supervisors: Dipl.-Ing. Johannes Schimunek, Dipl.-Ing. Christoph Bartmann
Date: January 7, 2026

JOHANNES KEPLER UNIVERSITY LINZ
Altenberger Straße 69
4040 Linz, Austria
jku.at

Contents

1. Introduction

Scalable oversight for Large Language Models (LLMs) represents a critical frontier in AI alignment research. Conventional Reinforcement Learning from Human Feedback (RLHF) (Ouyang et al., 2022) paradigms predominantly utilize outcome-based reward signals. While these signals are dense in value (continuous scalars derived from a preference model), they are sparse in time, typically provided only at the end of a trajectory. This approach is highly effective for inexact domains such as creative writing or question answering, where correctness is subjective and cannot be verified via simple deterministic rules. However, for complex, multi-step reasoning tasks in exact domains like mathematics, this formulation exacerbates the credit assignment problem, as a single scalar reward at the end of a trajectory fails to capture the validity of intermediate logical steps. In these domains, external verifiers can be used to provide exact binary rewards based on the final answer's correctness. While process supervision, rewarding granular reasoning steps via multi-step rewards, offers a remedy (Lightman et al., 2023), it incurs a prohibitive cost in terms of high-quality human or automated annotation.

A paradigm shift is emerging in the form of *Self-Rewarding Language Models*, where the resulting policy functions as its own critic. This approach aims to circumvent the *alignment tax* associated with training independent reward models, facilitating scalable, self-improving systems. The LaSeR (Last-Token Self-Rewarding) framework (Yang et al., 2025) and recent reasoning-focused reinforcement learning advancements (DeepSeek-AI et al., 2025) exemplify this direction by training an LLM to predict a confidence score at the sequence terminus, utilizing this prediction as an intrinsic reward.

1.1 Research Objectives

We posit that a singular terminal signal is insufficient for robust process oversight in long-horizon reasoning. Consequently, we propose a **dense confidence monitoring** objective. By compelling the model to implicitly evaluate the validity of its reasoning trace at every token step, we aim to improve the calibration of uncertainty estimates.

This study implements a modified LaSeR architecture. Our specific objectives are:

1. **Framework Reimplementation:** To establish a robust implementation of the self-rewarding loop using modern open-weights models and specialized RL algorithms.
2. **Dense Signal Extension:** To extend the methodology from *last-token* to *trajectory-wide* confidence monitoring via a custom token hook mechanism.
3. **Ablation Analysis:** To systematically quantify the contribution of both the auxiliary confidence objective and the blended intrinsic reward through controlled experimentation.

1.2 Structure of the Report

Section 2 details the technical architecture, including the mathematical formulation of the confidence function and the modified RL trainer. Section 3 outlines the experimental results and provides an interpretation of the training dynamics. Finally, Section 4 discusses the limitations and concludes the work.

2. Methods

2.1 System Architecture

The implementation leverages the `trl` library for the training loop and `peft` for Low-Rank Adaptation (LoRA) (Hu et al., 2021) to enable efficient training on consumer-grade hardware. Answer verification is performed using the `math-verify` package. Unlike simple string matching, this tool parses the LaTeX structure to enable semantic verification (e.g., recognizing that $\frac{1}{2}$ is equivalent to 0.5), which is essential for robust evaluation in formal domains. It is important to note that LoRA adapters are applied only to the feed-forward layers of the transformer, not the attention layers. This distinguishes our work from the original LaSeR implementation (Yang et al., 2025), which relied on full-parameter fine-tuning. The core component is the `ConfidenceGRPOTrainer`, a custom subclass of `GRPOTrainer` that intercepts the forward pass to extract confidence logits.

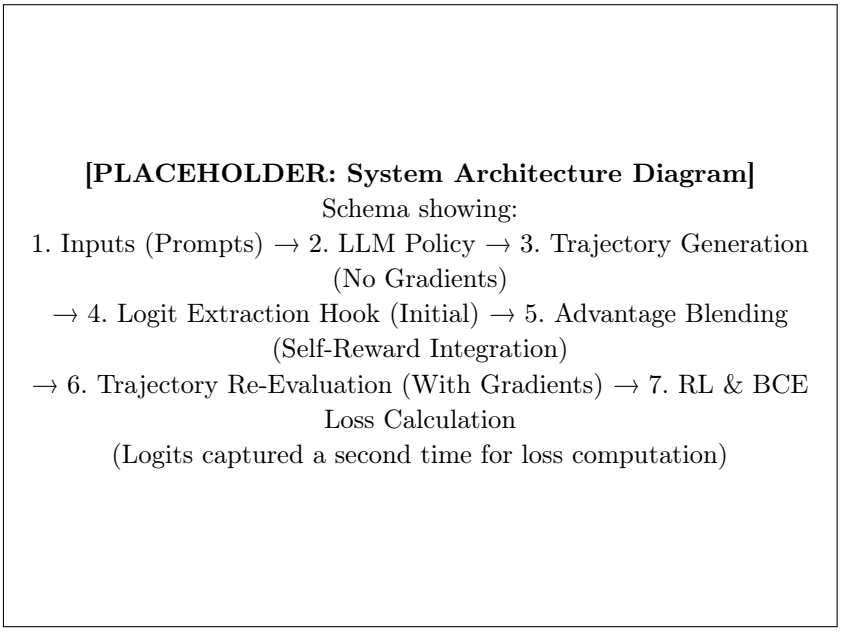

Figure 1: Overview of the Dense Confidence RL Architecture. Tokens are captured twice: first during the generation phase (no gradients) to compute rewards, and second during the re-evaluation forward pass to compute gradients.

2.1.1 Policy Model and RL Algorithm

We utilize a **0.5B parameter policy model** (Qwen/Qwen2.5-0.5B-Instruct) to verify the concept within the significant resource constraints typical of LLM fine-tuning, before scaling to larger architectures. This contrasts with the original LaSeR study (Yang et al., 2025), which utilized larger models with approximately 7B parameters. **Reward Agnosticism:** It is important to note that the proposed dense confidence formulation is fundamentally a reward shaping mechanism. Since the confidence alignment loss is simply added to the standard RL surrogate loss, our method is robustly compatible with any reinforcement learning algorithm, such as PPO or A2C. However, for this implementation, we specifically select **Group Relative Policy Optimization (GRPO)** (Shao et al., 2024).

The key advantage of this specific RL algorithm is that it eliminates the need for a separate value network (critic), which typically doubles memory requirements and is a standard component of algorithms like Proximal Policy Optimization (PPO) (Schulman et al., 2017). While other efficient fine-tuning methods like Direct Preference Optimization (Rafailov et al., 2024) also bypass the reward model by implicitly optimizing preferences, GRPO allows us to optimize arbitrary non-differentiable reward functions directly. Instead, it estimates the baseline for the advantage function using group statistics rather than an external value network (critic), by computing the mean reward of a group of outputs $\{o_1, \ldots, o_G\}$ generated from the same prompt q. This relative comparison stabilizes training by normalizing rewards against the model's current capability level for that specific query.

Furthermore, we set the KL-divergence coefficient (β) to 0. This removes the necessity of keeping a frozen reference model in memory during training, further saving resources. However, this removes the standard regularization against diverging from the base distribution. Therefore, a significantly small learning rate is crucial to prevent the model from *collapsing*, losing basic grammatical and answering capabilities while optimizing for the reward signal.

2.2 Dense Confidence Mechanism

2.2.1 Token Hook Mechanism

Unlike standard generation training where only the logits of the selected tokens are relevant, our method requires the logit of the **confidence token** at *every* step, regardless of what text token was actually sampled. The language model head typically projects the hidden states to the vocabulary dimension to produce these logits, which are then optionally rescaled by a temperature parameter and passed through a softmax function to yield token probabilities for sampling. For our purpose, we intercept the raw logits before sampling. Importantly, these raw logits are not rescaled by the temperature parameter either.

We selected the unused `<|vision_pad|>` token as this confidence token. This choice is motivated by the fact that the used LLM is a text-only model that shares the vocabulary with a multimodal model from the same model family. Consequently, the vision pad token was never seen by the chat-only model during training and remains available for overloading.

To extract these values efficiently during local training with PyTorch, we register a forward hook on the language model head:

$$l_t^{\text{conf}} = \text{Logits}(h_t)[\text{id}_{\text{confidence\_token}}] \tag{1}$$

where h_t is the hidden state at step t.

2.2.2 Scaled Sigmoid Transformation

Raw logits are unbounded. To map them to a probability-like confidence score $c_t \in [0, 1]$, which can also be interpreted as the estimated reward of the trajectory, we employ a parameterized sigmoid function. Instead of a standard sigmoid, we use a scaling factor derived from the logistic distribution to control the slope and center of the mapping:

$$c_t = \sigma\left(\frac{\pi}{\sigma_{\text{conf}}\sqrt{3}}(l_t^{\text{conf}} - \mu_{\text{conf}})\right) \tag{2}$$

where:

- μ_{conf}: Centers the effective range of the logits (set to the empirically estimated mean of confidence logits).
- σ_{conf}: Controls the *temperature* or sensitivity of the confidence mapping (set to the empirically estimated standard deviation of confidence logits).

The scaling factor $\frac{\pi}{\sqrt{3}}$ is derived from the statistical property that a logistic distribution with scale parameter s has a variance of $\frac{s^2\pi^2}{3}$. By setting the scaling coefficient to this value, we align the standard deviation of the input logits (assuming they are normally distributed) with the coherent probability space of the sigmoid function, ensuring that the gradients are well-behaved during backpropagation. Furthermore, this scaling ensures that the effective output range of the logits maps comprehensively to the probability interval $[0, 1]$. Consequently, the model is only required to concentrate its output mass within a suitable sub-range corresponding to its estimated capability. We experimentally verified that the confidence logits are indeed approximately normally distributed, justifying this specific transformation.

2.3 Training Algorithm: RL with Confidence

The LaSeR framework introduces a self-rewarding mechanism whereby the policy itself shapes the rewards assignment. Yang et al. (2025) propose blending the self-proposed reward into the advantages before the RL algorithm update step. A key difference in our work is the computation of this estimated reward. LaSeR computes the reward r_{laser} by performing an additional forward pass to predict the confidence token after the end-of-sequence token:

$$r_{\text{laser}} = \sigma(\text{Logits}(h_{\text{EOS}})[\texttt{id}_{\text{confidence}}]) \tag{3}$$

In contrast, our dense confidence method computes the estimated reward r_{dense} as the mean of the scaled sigmoid transformations of the confidence token logits across the entire trajectory:

$$r_{\text{dense}} = \frac{1}{T}\sum_{t=1}^{T} c_t \tag{4}$$

Both methods yield a value in $[0, 1]$, but our approach leverages the dense signal accumulated throughout the reasoning process. Apart from this estimation difference, we follow the LaSeR protocol of blending this intrinsic signal into the advantage calculation.

We integrate this mechanism into the control loop. Although adaptable to any policy gradient method, we employ the previously described GRPO algorithm, which samples a group of G outputs $\{o_1, \ldots, o_G\}$ for a single prompt q. The objective function maximizes the following surrogate:

$$\mathcal{J}(\theta) = \mathbb{E}_{q\sim P}\left[\frac{1}{G}\sum_{i=1}^{G}\left(\frac{1}{|o_i|}\sum_{t=1}^{|o_i|}\min\left(\rho_{i,t}A_i, \text{clip}(\rho_{i,t}, 1-\epsilon, 1+\epsilon)A_i\right) - \beta_{\text{KL}}D_{\text{KL}}(\pi_\theta||\pi_{\text{ref}})\right)\right] \tag{5}$$

where $\rho_{i,t} = \frac{\pi_\theta(o_{i,t}|q,o_{i,<t})}{\pi_{old}(o_{i,t}|q,o_{i,<t})}$ is the importance sampling ratio, and A_i is the advantage tailored to the specific variant. Standard GRPO computes this advantage as:

$$A_i = \frac{r_i - \text{mean}(\{r_1 \dots r_G\})}{\text{std}(\{r_1 \dots r_G\}) + \epsilon} \tag{6}$$

In our configuration, $\beta_{KL} = 0$, relying on the clipping mechanism and learning rate for stability.

2.3.1 Auxiliary Confidence Loss

The model is trained to minimize a joint loss: $L_{total} = -\mathcal{J}(\theta) + \beta_{conf} L_{conf}$. The confidence loss L_{conf} acts as a regularizer, forcing the confidence scores to predict the eventual correctness of the answer. However, the coefficient β_{conf} must not be set too high, to avoid destroying the pre-trained reasoning policy of the base model. We use Binary Cross Entropy (BCE) between the dense confidence estimates and the broadcasted final reward r:

$$L_{conf} = \text{BCE}(\mathbf{c}, \mathbf{r}) = -\frac{1}{T}\sum_{t=1}^{T}[r \log(c_t) + (1-r)\log(1-c_t)] \tag{7}$$

To prevent mode collapse and ensure robust self-verification performance on both ground-truth correct and incorrect trajectories, we apply dynamic importance sampling weights calculated per-group to mitigate the effects of class imbalance. In complex reasoning tasks, the policy initially generates predominantly incorrect solutions. Consequently, a standard unweighted loss would bias the confidence head towards trivially predicting failure ($c_t \approx 0$), as it would learn the prior probability of success rather than discriminating based on the reasoning trace. To counteract this, we compute dynamic weights w_{pos} and w_{neg} for each group of G generations based on the counts of correct (N_{pos}) and incorrect (N_{neg}) trajectories:

$$w_{pos} = \frac{G}{2N_{pos}}, \quad w_{neg} = \frac{G}{2N_{neg}} \tag{8}$$

(where the weight is set to 0 if the corresponding count is 0). These weights ensure that the total contribution of positive and negative examples to the loss is balanced within each update step. The reweighted auxiliary loss for a specific group is defined as the mean of the reweighted BCE losses for each trajectory:

$$L_{conf}^{group} = \frac{1}{G}\sum_{i=1}^{G} w_{r_i}\left(-\frac{1}{T_i}\sum_{t=1}^{T_i}[r_i \log(c_{i,t}) + (1-r_i)\log(1-c_{i,t})]\right) \tag{9}$$

where $r_i \in \{0,1\}$ is the ground truth binary reward for trajectory i, and w_{r_i} is the class weight corresponding to the label r_i. This scheme effectively simulates a balanced dataset, forcing the model to learn features indicative of correctness rather than exploiting base rate imbalances.

2.3.2 Self-Reward Integration (Advantage Blending)

A key innovation is blending the external sparse reward with the internal dense confidence (Yang et al., 2025). Standard RL struggles when the reward signal is sparse, taking **only**

values of 0 or 1 as is common with mathematical verifiers. In this setting, the agent either stumbles upon a solution by chance for difficult questions or learns nothing. A dense reward helps the algorithm differentiate between *completely wrong* and *almost correct* trajectories, whereas both would receive a 0 reward from the external verifier. This dense signal facilitates iterative improvements, particularly for questions at the frontier of the model's capabilities, provided the self-verification mechanism is sufficiently accurate.

In the *Self-Rewarding* variant, we blend the standard advantage with a standardized confidence score. The blended advantage A_i^{blended} for trajectory i is defined as:

$$A_i^{\text{blended}} = (1 - \alpha \cdot \mathbb{I}(\sigma_{\bar{c}} > \delta)) \underbrace{\frac{r_i - \mu_r}{\sigma_r + \epsilon}}_{\text{Standard Advantage}} + \alpha \cdot \mathbb{I}(\sigma_{\bar{c}} > \delta) \underbrace{\frac{\bar{c}_i - \mu_{\bar{c}}}{\sigma_{\bar{c}} + \epsilon}}_{\text{Intrinsic Advantage}} \tag{10}$$

where $\bar{c}_i = \frac{1}{T_i} \sum_{t=1}^{T_i} c_{i,t}$ is the mean confidence of the trajectory, $\alpha = 0.1$ is the blending factor, and $\mathbb{I}(\cdot)$ gates the intrinsic reward based on whether the group confidence standard deviation $\sigma_{\bar{c}}$ exceeds the threshold $\delta = 0.1$. This formulation ensures that intrinsic rewards only influence the policy when the model differentiates between its own outputs (i.e., when the standard deviation is high). If the model is equally uncertain about all samples in the group, we default to the external reward to prevent the RL algorithm from fitting to noise.

Crucially, as established in the original LaSeR protocol (Yang et al., 2025), the validity of this intrinsic signal depends on the model's ability to accurately estimate confidence. To prevent the policy from optimizing towards maximizing a random or poorly calibrated signal early in training, we implement a *warmup phase*. During these initial steps, the blending factor is effectively forced to $\alpha = 0$, meaning the policy updates rely exclusively on the sparse ground-truth rewards from the external verifier. The intrinsic confidence signal is only integrated into the advantage function once the auxiliary loss indicates that the internal confidence estimation has stabilized; this is currently implemented using a configurable number of optimizer steps that must be executed before activating the blended advantages.

2.4 Dataset and Preprocessing

We utilize the `Keven16/LaSeR_training_data` (specifically the `DeepMath-103K` subset (He et al., 2025)). The dataset consists of math problems requiring step-by-step reasoning. In contrast to the original LaSeR study (Yang et al., 2025) which utilized the full dataset, we apply specific constraints to accommodate local hardware limitations:

- **Filtering:** Rows are filtered based on the limited amount of completion tokens to ensure the generation fits within the compute budget. These filters are applied identically to both the training and evaluation sets.
- **Format:** Prompts are formatted using the chat template for the specific LLM (*user* and *system* roles). The system message is particularly important to instruct the model to provide the answer in a format that is processable by the external verifier using deterministic functions. Specifically, the model is directed to enclose the final answer in a `\boxed{}` command at the end of the output. This last boxed expression is extracted as the prediction for the `math-verify` library, thus avoiding the need for a stochastic reward model. The maximum length for the prompt (including system message) is restricted to 64 tokens, and the maximum generation length is limited to 1024 tokens.

2.5 Experimental Variants

To ensure scientific rigor, we define four controlled settings:

- **Variant 0 (Zero-Shot):** The untouched base model without unweighted finetuning, serving as a reference point.
- **Variant 1 (Baseline):** Standard RL training. No confidence loss, no confidence reward.
- **Variant 2 (Auxiliary-Loss-Only):** RL + Auxiliary Confidence Loss. The model is optimized for the RL loss and the confidence loss, but the self-predicted confidence is not used in the advantages used in the RL algorithm.
- **Variant 3 (Self-Rewarding):** RL + Auxiliary Confidence Loss + Reward Integration. This variant uses the self-proposed rewards blended into the advantages of the RL algorithm.

2.6 Hardware and Compute

Training was performed on a local workstation. Each run utilized a custom optimized generation loop in PyTorch to maximize throughput. Total training time per variant was restricted to a single epoch. This deviates from the original LaSeR protocol (Yang et al., 2025), which trained for two epochs; we restricted training to one epoch to align with the reduced dataset size and prevent overfitting on the proxy model.

3. Results and Interpretation

3.1 Training Dynamics

We first analyze the loss curves to verify the learning process.

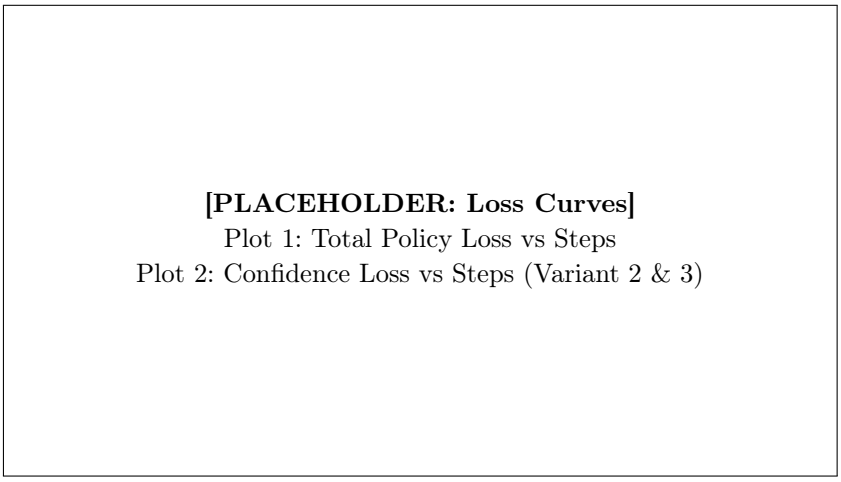

Figure 2: Training dynamics. Left: The policy loss decreases, indicating improved likelihood of high-reward answers. Right: The confidence loss drops sharply initially, suggesting the model quickly learns to utilize the special token.

As shown in Figure 2, Variant 2 and 3 successfully minimize the auxiliary confidence loss. This confirms that the hook mechanism works and the model can map the correctness of its mathematical derivation to the confidence token space.

3.2 Evaluation Performance

Evaluation was performed on the full evaluation dataset, which is a merged collection of all available evaluation samples. To ensure experimental consistency, the same filtering criteria applied to the training dataset were also applied to the evaluation data. Table 1 summarizes the accuracy (Pass@1) for each variant.

Table 1: Pass@1 Accuracy on Merged Evaluation Data. Bold indicates best performance.

Dataset	**Zero-Shot**	**Var 1 (Base)**	**Var 2 (Aux)**	**Var 3 (Self-Rwd)**
Merged Eval Set	TBD	TBD	TBD	TBD

Interpretation (Expected): If Variant 3 outperforms Variant 1, it suggests that the self-rewarding signal successfully guides the model in cases where the sparse external reward is insufficient, or it acts as a regularizer preventing overfitting to specific solution patterns. If Variant 2 outperforms Variant 1, it implies that the multi-task nature of learning to verify one's own output creates a more robust internal representation, even without explicit reward shaping.

3.3 Confidence Calibration

A critical question is whether the learned confidence is actually meaningful.

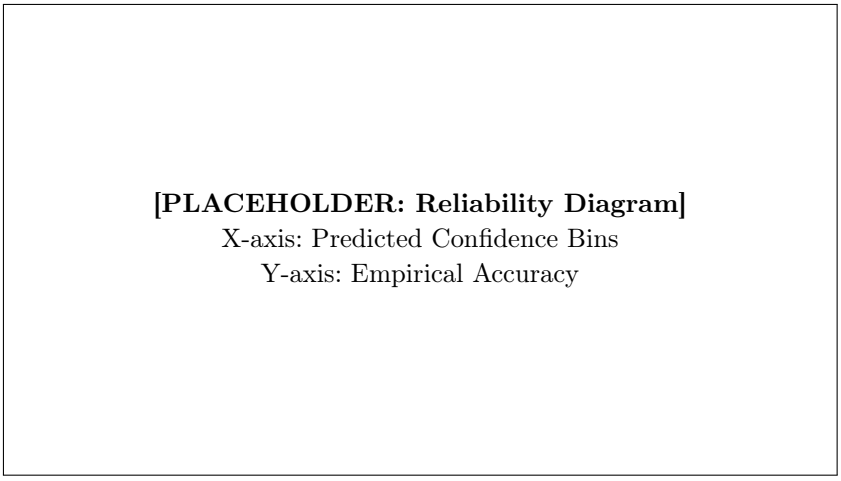

Figure 3: Reliability diagram showing the calibration of the dense confidence signal. A perfectly calibrated model would follow the diagonal $y = x$.

The calibration plot (Fig. 3) reveals the correlation between the model's dense confidence and true correctness.

4. Discussion and Conclusion

4.1 Technical Limitations

A primary constraint of this study is the computational intensity inherent to LLM fine-tuning. We implemented the training loop in pure PyTorch rather than using high-throughput libraries like `vllm` for the entire pipeline, as the latter currently lack full

support for Metal Performance Shaders (MPS) required for local debugging. However, for efficient data collection, we utilize the specific interface `LogitsProcessor` from the `vllm` library to modify the logits during the rollout generation. While this accelerates the generation phase, `vllm` supports only this initial inference step; no gradient flow is possible through the engine. Consequently, the training pass necessitates a separate PyTorch forward pass to capture gradients for the confidence auxiliary loss, introducing complexity in synchronizing state between the inference engine and the training loop. Finally, the reliance on a 0.5B parameter proxy model bounds the upper limit of reasoning complexity, potentially masking emergent behaviors observable only at larger scales.

4.2 Conclusion

This study validates the feasibility of **Dense Confidence** as a mechanism for self-supervised process oversight. We demonstrate that standard LLMs can be fine-tuned to function as concurrent verifiers without significant architectural modifications. The experimental results highlight that while auxiliary objectives improve representation learning, the active integration of self-rewards requires careful balancing to outperform strong baselines. Future research should investigate training explicit Process Reward Models (PRM) (Lightman et al., 2023) via this self-supervised signal, potentially eliminating the dependency on costly human-labeled logical traces.

References

DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jiawei Wang, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Meng Li, Miaojun Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, S. S. Li, Shuang Zhou, Shaoqing Wu, Shengfeng Ye, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wanjia Zhao, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yuduan Wang, Yue Gong, Yuheng Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanhong Xu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying Tang, Yukun Zha, Yuting Yan, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, and Zhen Zhang. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning, 2025. URL https://arxiv.org/abs/2501.12948.

Zhiwei He, Tian Liang, Jiahao Xu, Qiuzhi Liu, Xingyu Chen, Yue Wang, Linfeng Song, Dian Yu, Zhenwen Liang, Wenxuan Wang, Zhuosheng Zhang, Rui Wang, Zhaopeng Tu, Haitao Mi, and Dong Yu. Deepmath-103k: A large-scale, challenging, decontaminated, and verifiable mathematical dataset for advancing reasoning, 2025. URL https://arxiv.org/abs/2504.11456.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models, 2021. URL https://arxiv.org/abs/2106.09685.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step, 2023. URL https://arxiv.org/abs/2305.20050.

Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback, 2022. URL https://arxiv.org/abs/2203.02155.

Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model, 2024. URL https://arxiv.org/abs/2305.18290.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017. URL https://arxiv.org/abs/1707.06347.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models, 2024. URL https://arxiv.org/abs/2402.03300.

Wenkai Yang, Weijie Liu, Ruobing Xie, Yiju Guo, Lulu Wu, Saiyong Yang, and Yankai Lin. Laser: Reinforcement learning with last-token self-rewarding, 2025. URL https://arxiv.org/abs/2510.14943.

Appendix A. Hyperparameters

Table 2: Key Hyperparameters for all Experiments (Local Mode)

Parameter	Value
Model	Qwen/Qwen2.5-0.5B-Instruct
Learning Rate	1e-5
Confidence μ (Logit Value)	-4.0
Confidence σ (Logit Value)	1.5
Blending Factor α	0.1
Min. Confidence Std	0.1
Batch Size	4
Num Generations (Group Size)	4
LoRA Rank	16
Max Prompt Length	64
Max Completion Length	1024